Requirements Specification
Medical Delivery Drone REST API

Software Testing Portfolio - Supporting Document

1 Overview

This document specifies the requirements for the Medical Delivery Drone REST API, derived from the ILP coursework specification (CW1/CW2). These requirements form the basis for the testing strategy documented in the main portfolio.

2 Requirements Summary

Requirement Type	Count	Examples
Functional	12	Distance calculation, path planning, drone queries
Performance	2	30s timeout, response time under load
Robustness	3	Invalid input handling, error responses, edge cases
Safety	2	No-fly zone avoidance, battery limit enforcement
Security	2	Input validation, injection prevention
Reliability	1	Fresh data (no stale caching)
Availability	1	Docker container health check
Qualitative	3	Maintainability, testability, interoperability
Total	**26**	

3 Functional Requirements

3.1 CW1 – Basic Drone Operations (5 requirements)

ID	Endpoint	Description
FR-01	`/distanceTo`	Compute the Euclidean distance between two geographic positions: $d = \sqrt{(lng_1 - lng_2)^2 + (lat_1 - lat_2)^2}$
FR-02	`/isCloseTo`	Determine whether two positions are within the threshold of 0.00015 degrees of each other
FR-03	`/nextPosition`	Calculate the next position for a drone given a starting position and a direction angle from a 16-point compass rose with a fixed move distance of 0.00015 degrees
FR-04	`/isInRegion`	Determine whether a given position lies within a defined geographic polygon region using Java's Path2D.contains() method
FR-05	`/uid`	Return the student's unique identifier string

3.2 CW2 – Drone Fleet Management & Path Planning (7 requirements)

ID	Endpoint	Description

FR-06	`/dronesWithCooling/{state}`	Return drone IDs matching the cooling capability requirement
FR-07	`/dronesWithHeating/{state}`	Return drone IDs matching the heating capability requirement
FR-08	`/droneDetails/{id}`	Retrieve the complete specification for a single drone (capacity, battery, temperature control capabilities)
FR-09	`/query`	Filter drones by attribute-value pairs provided in the request body
FR-10	`/queryAvailableDrones`	Find drones suitable for a list of medication dispatch requests based on weight, temperature, and availability constraints
FR-11	`/calcDeliveryPath`	Match medications to appropriate drones, calculate optimal delivery sequence using A* pathfinding, avoid all no-fly zones, ensure drones return to starting service point, respect battery/move limits (`maxMoves`), return flight path with total cost and moves
FR-12	`/calcDeliveryPathAsGeoJson`	Export the delivery path as a valid GeoJSON LineString for visualization

4 Performance Requirements

ID	Requirement	Measurable Criteria
PR-01	Response Time	Any individual endpoint call (including route calculation) must complete within **30 seconds**
PR-02	Workload Handling	Path calculation must handle 50+ delivery requests within the time limit under realistic workloads

5 Robustness Requirements

ID	Requirement	Expected Behaviour
RB-01	Invalid Input Handling	Return `400 Bad Request` for syntactically incorrect input (malformed JSON, wrong types) rather than crashing
RB-02	No-Match Scenarios	Return `200 OK` with empty result for semantically valid but no-match scenarios
RB-03	Edge Case Handling	Graceful handling of null values, empty lists, and boundary coordinates (± 180 longitude, ± 90 latitude)

6 Safety Requirements

ID	Requirement	Constraint
SF-01	No-Fly Zone Compliance	Generated flight paths must **never** cross or enter no-fly zone boundaries

SF-02	Battery Limit Enforcement	Drones must never be assigned paths exceeding their `maxMoves` battery capacity

7 Security Requirements

ID	Requirement	Implementation
SC-01	Input Validation	All user-supplied coordinates, IDs, and JSON bodies must be validated before processing
SC-02	Injection Prevention	No direct string concatenation in queries; parameterised parsing only

8 Reliability Requirements

ID	Requirement	Constraint
RL-01	Data Freshness	The system must accurately reflect the current state of the environment (drone availability, restricted areas) by fetching data from the external ILP-REST-service on every endpoint call - no stale caching is permitted

9 Availability Requirements

ID	Requirement	Constraint
AV-01	Health Check	The service must consistently respond with `status: UP` at the `/actuator/health` endpoint
AV-02	Containerisation	The complete system must run in a Docker container exposing port 8080 for the Java REST service

10 Qualitative Requirements

ID	Requirement	Description
QL-01	Maintainability	The codebase should follow clean architecture principles, separating controllers, services, and data access layers to facilitate future enhancements
QL-02	Testability	The system should be designed with dependency injection to allow mocking of external services (e.g., `ILP_ENDPOINT`) during unit and integration testing
QL-03	Interoperability	The service must correctly consume the `ILP_ENDPOINT` environment variable to connect to the central ILP-REST data service and parse its JSON responses

11 Requirements Traceability

Each requirement is traced to test cases in the main test suite (JUnit) and Postman end-to-end tests:

Requirement ID	JUnit Test Class	Postman Tests	Coverage
FR-01 to FR-05	`PositionTest`, `RegionTest`	4 tests	Unit + E2E
FR-06 to FR-10	`DroneServiceImplMockTest`	15+ tests	Unit + E2E
FR-11, FR-12	`DeliveryPathPerformanceTest`	14 tests	Integration + E2E
PR-01, PR-02	`DeliveryPathPerformanceTest`	All tests	Performance
RB-01 to RB-03	`DroneApiContractEdgeCaseTest`	2+ tests	Contract + E2E
SF-01, SF-02	`DroneServiceImplMockTest`	GeoJSON tests	Integration + E2E
SC-01, SC-02	`DroneApiContractTest`	Implicit	Contract
RL-01	(Mocked in JUnit)	All E2E tests	E2E only
AV-01	`IlpCw1ApplicationTests`	Health check	Integration + E2E
QL-01 to QL-03	(Architecture review)	N/A	Manual

12 Postman End-to-End Test Suite

The following Postman tests validate all endpoints against the live service:

#	Endpoint	Description
1	`GET /actuator/health`	Health check returns `status: UP`
2	`GET /api/v1/uid`	Returns student identifier
3	`POST /api/v1/distanceTo`	Euclidean distance calculation
4	`POST /api/v1/isCloseTo`	Proximity threshold check
5	`POST /api/v1/nextPosition`	Compass rose position calculation
6	`POST /api/v1/isInRegion`	Polygon containment check
7–8	`GET /dronesWithCooling/{true|false}`	Filter drones by cooling capability
9	`GET /droneDetails/4`	Valid drone ID (1–10)
10	`GET /droneDetails/11`	Invalid drone ID (error handling)
11–17	`POST /calcDeliveryPath1--7`	Path calculation with 1–8 deliveries
18–24	`POST /query1--7`	Drone attribute queries (capacity, heating, maxMoves)
25–29	`POST /queryAvailableDrones1--5`	Available drones for delivery combinations
30–34	`GET /queryAsPath/{attr}/{val}`	Path-based queries (cooling, capacity, maxMoves, costPerMove, id)
35–41	`POST /calcDeliveryPathAsGeoJson1--7`	GeoJSON output for no-fly zone verification

Total: 40+ end-to-end tests covering all functional requirements with real service integration.